RECURSOS

SESSIÓ 6 · SERIALITZACIÓ D'OBJECTES I JSON AMB JACKSON

Correspondència amb les RA: RA1. Desenvolupa aplicacions que gestionen informació emmagatzemada en fitxers identificant-ne el camp d'aplicació i utilitzant classes específiques. CA 1.4, 1.5 i 1.6.

Dimecres 30/09/2026 · DAM2 · sessió 6 de 58 · AEA1 «Gestió de fitxers amb Java» · AA3 «Canvia de format» (1a part)

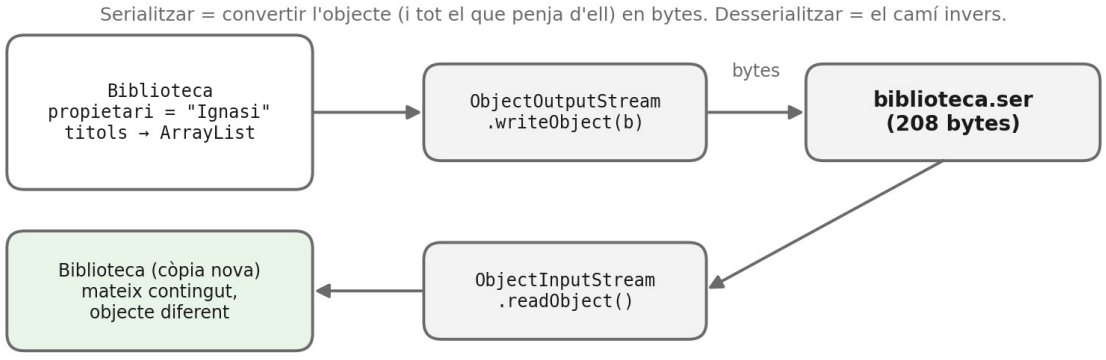
La sessió d'avui en un cop d'ull

Fins ara hem guardat **dades**: línies de text, números, camps d'un CSV. Avui guardem **objectes sencers**. Ho farem de dues maneres que semblen iguals i no ho són gens: la serialització pròpia de Java (`ObjectOutputStream`), que és còmoda però només la llegeix Java i té riscos, i JSON amb la biblioteca **Jackson**, que és el format d'intercanvi que fan servir les API web. Amb això tindràs els punts 1, 2 i 3 de l'AA3; l'XML amb JAXB i el convertidor universal queden per a la sessió 7.

Els exemples són al repositori del mòdul, paquet `cat.pratfp.gamevault.sessio6`: **github.com/ilopez000/0486-acces-a-dades**. Fes `git pull`.

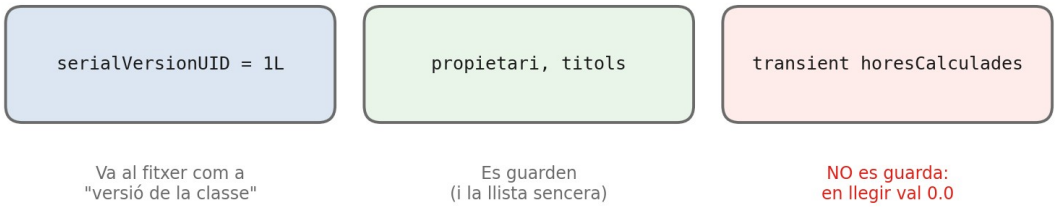
> **El que t'has d'endur**
> Serialitzar és convertir un objecte (i tot el graf que penja d'ell) en bytes; desserialitzar és reconstruir-lo.
> `serialVersionUID` diu quina versió de la classe hi ha al fitxer; `transient` marca el que no s'ha de guardar.
> No desserialitzis mai un fitxer d'origen desconegut sense un `ObjectInputFilter`.
> Jackson converteix objectes ↔ JSON amb un `ObjectMapper` ben configurat: `JavaTimeModule` per a les dates és obligatori.

1. Què és serialitzar



El cicle complet: l'objecte es converteix en bytes, els bytes es guarden, i més tard se'n reconstrueix una còpia.

Perquè una classe es pugui serialitzar només ha d'implementar la interfície `Serializable`, que no té cap mètode: és una **marca**. Java s'encarrega de recórrer tots els camps de l'objecte i escriure'ls, incloent-hi els objectes als quals apunta (una llista, per exemple). Hi ha dues paraules que has de conèixer:



Els tres tipus de camp d'una classe serialitzable.

- **`serialVersionUID`**: un número de versió de la classe que viatja dins del fitxer. Si la classe canvia i el número també, Java es nega a llegir fitxers antics (`InvalidClassException`). Declara'l sempre explícitament; si no ho fas, Java en calcula un a partir de l'estructura de la classe, i qualsevol canvi el trenca sense avisar.
- **`transient`**: el camp no es guarda. Serveix per a dades derivades (que es poden recalcular), per a connexions o fitxers oberts (que no tenen sentit en un altre moment) i per a dades que no vols que surtin de la memòria (contrasenyes).

2. Exemple 1 · Desar i recuperar una Biblioteca

Primer, la classe que serialitzarem. És la mateixa `Biblioteca` que demana el punt 2 de l'AA3: propietari, llista de títols i un camp `transient`.

src/main/java/cat/pratfp/gamevault/sessio6/Biblioteca.java

```
 1  package cat.pratfp.gamevault.sessio6;
 2
 3  import java.io.Serializable;
 4  import java.util.ArrayList;
 5  import java.util.List;
 6
 7  /** La biblioteca d'un jugador. Es pot serialitzar: es converteix en bytes i
es recupera sencera. */
 8  public class Biblioteca implements Serializable {
 9
10      // Número de versió de la classe. Si canvia, els fitxers antics deixen
de ser compatibles.
11      private static final long serialVersionUID = 1L;
12
13      private final String propietari;
14      private final List<String> titols = new ArrayList<>();
15      // transient: aquest camp NO es guarda al fitxer. Es recalcula quan cal.
16      private transient double horesCalculades;
17
18      public Biblioteca(String propietari) {
19          this.propietari = propietari;
20      }
21
22      public void afegeix(String titol, double hores) {
23          titols.add(titol);
24          horesCalculades += hores;
25      }
26
27      public String getPropietari() { return propietari; }
28      public List<String> getTitols() { return titols; }
29      public double getHoresCalculades() { return horesCalculades; }
30
31      @Override
32      public String toString() {
33          return propietari + " té " + titols + " (" + horesCalculades + "
h)";
34      }
35  }
```

Línia	Codi	Què fa i per què
8	`public class Biblioteca implements Serializable {`	`implements Serializable`: la marca. Sense això, `writeObject` llança `NotSerializableException`.
11	`private static final long serialVersionUID = 1L;`	El número de versió, explícit. `1L` és la primera versió. `private static final long` és la firma exacta que Java busca.
13-14	`private final String propietari;` `private final List<S…`	Camps normals: es guarden. La llista també, amb tot el seu contingut (les cadenes són `Serializable`).
16	`private transient double horesCalculades;`	`transient`: les hores calculades **no** van al fitxer. En llegir, el camp tindrà el valor per defecte (`0.0`).
22-25	`public void afegeix(String titol, double hores) {` …	Cada vegada que afegim un joc, sumem hores. És una dada derivada: es podria recalcular a partir d'una altra font.
32-34	`public String toString() {` `return propietari + " …`	`toString` ens permet veure d'un cop d'ull què conté l'objecte abans i després del viatge.

src/main/java/cat/pratfp/gamevault/sessio6/Ex1Serialitza.java

```
 1  package cat.pratfp.gamevault.sessio6;
 2
 3  import java.io.IOException;
 4  import java.io.ObjectInputStream;
 5  import java.io.ObjectOutputStream;
 6  import java.nio.file.Files;
 7  import java.nio.file.Path;
 8
 9  /** Exemple 1: desar un objecte sencer amb ObjectOutputStream i recuperar-lo
amb ObjectInputStream. */
10  public class Ex1Serialitza {
11
12      public static void main(String[] args) throws IOException,
ClassNotFoundException {
13          Path fitxer = Path.of("dades", "biblioteca.ser");
14
15          // 1. CREAR L'OBJECTE en memòria.
16          Biblioteca original = new Biblioteca("Ignasi");
17          original.afegeix("Hollow Knight", 42.5);
18          original.afegeix("Gris", 4.0);
19          System.out.println("Abans:   " + original);
20
21          // 2. SERIALITZAR: l'objecte i tot el que penja d'ell (la llista) es
converteixen en bytes.
22          try (ObjectOutputStream out = new
ObjectOutputStream(Files.newOutputStream(fitxer))) {
23              out.writeObject(original);
24          }
25          System.out.println("Desat a " + fitxer + " (" + Files.size(fitxer) +
" bytes)");
26
27          // 3. DESSERIALITZAR: es reconstrueix un objecte nou a partir dels
bytes.
28          Biblioteca recuperada;
29          try (ObjectInputStream in = new
ObjectInputStream(Files.newInputStream(fitxer))) {
30              recuperada = (Biblioteca) in.readObject();
31          }
32          System.out.println("Després: " + recuperada);
33          System.out.println("És el mateix objecte? " + (original ==
recuperada));
34      }
35  }
```

Línia	Codi	Què fa i per què
12	`public static void main(String[] args) throws IOException…`	`readObject` pot llançar `ClassNotFoundException` (si la classe del fitxer no existeix al programa). Cal declarar-la o capturar-la.
13	`Path fitxer = Path.of("dades", "biblioteca.ser");`	Per convenció, els fitxers serialitzats porten extensió `.ser`.
16-18	`Biblioteca original = new Biblioteca("Ignasi");` o…	Creem l'objecte i hi afegim dos jocs: 46,5 hores calculades.
22-24	`try (ObjectOutputStream out = new ObjectOutputStream(File…`	`ObjectOutputStream` embolcalla un flux de bytes. `writeObject` escriu l'objecte i tot el que penja d'ell. El `try` el tanca.

Línia	Codi	Què fa i per què
25	`System.out.println("Desat a " + fitxer + " (" + Files.siz…`	208 bytes per a un objecte tan petit: el format inclou el nom de la classe, el `serialVersionUID` i la descripció de cada camp.
29-31	`try (ObjectInputStream in = new ObjectInputStream(Files.n…`	`readObject` retorna un `Object`: cal fer el «cast» a `Biblioteca`. Si el fitxer no conté una `Biblioteca`, aquest «cast» peta amb `ClassCastException`.
32	`System.out.println("Després: " + recuperada);`	Els títols hi són; les hores, no: `0.0`. És l'efecte del `transient`.
33	`System.out.println("És el mateix objecte? " + (original =…`	`==` compara referències: és un objecte **nou** amb el mateix contingut. La igualtat de contingut la decidiria un `equals` (que aquí no hem escrit).

Sortida esperada

```
Abans:   Ignasi té [Hollow Knight, Gris] (46.5 h)
Desat a dades/biblioteca.ser (208 bytes)
Després: Ignasi té [Hollow Knight, Gris] (0.0 h)
És el mateix objecte? false
```

Per a l'AA3

Els mètodes `desa(Path)` i `carrega(Path)` que demana el punt 2 són exactament les línies 22-24 i 29-31, posades dins de la classe `Biblioteca`. Pensa què ha de retornar `carrega`: és un mètode `static` que crea una `Biblioteca` nova.

3. Exemple 2 · L'experiment del serialVersionUID

Què passa amb els fitxers `.ser` que vas desar l'any passat quan la classe canvia? L'exemple ho reprodueix en un sol programa: desa amb una versió de la classe i intenta llegir amb una altra.

src/main/java/cat/pratfp/gamevault/sessio6/Ex2VersioIncompatible.java

```
 1  package cat.pratfp.gamevault.sessio6;
 2
 3  import java.io.IOException;
 4  import java.io.InvalidClassException;
 5  import java.io.ObjectInputStream;
 6  import java.io.ObjectOutputStream;
 7  import java.io.Serializable;
 8  import java.nio.file.Files;
 9  import java.nio.file.Path;
10
11  /** Exemple 2: què passa quan la classe canvia després d'haver desat el
fitxer. */
12  public class Ex2VersioIncompatible {
13
14      /** Versió 1 de la classe, tal com es va desar «l'any passat». */
15      static class PerfilV1 implements Serializable {
16          private static final long serialVersionUID = 1L;
17          String nom = "Ignasi";
18      }
19
20      /** Versió 2: hem afegit un camp i hem canviat el número de versió. */
21      static class PerfilV2 implements Serializable {
22          private static final long serialVersionUID = 2L;
23          String nom = "Ignasi";
24          int nivell = 7;
25      }
26
27      public static void main(String[] args) throws IOException {
28          Path fitxer = Path.of("dades", "perfil.ser");
29
30          // 1. Desem amb la versió 1...
31          try (ObjectOutputStream out = new
ObjectOutputStream(Files.newOutputStream(fitxer))) {
32              out.writeObject(new PerfilV1());
33          }
34
35          // 2. ...i fem trampa: reescrivim el nom de la classe dins del
fitxer perquè apunti a la V2.
36          byte[] bytes = Files.readAllBytes(fitxer);
37          String contingut = new String(bytes,
java.nio.charset.StandardCharsets.ISO_8859_1);
38          contingut = contingut.replace("PerfilV1", "PerfilV2");
39          Files.write(fitxer,
contingut.getBytes(java.nio.charset.StandardCharsets.ISO_8859_1));
40
41          // 3. Intentem llegir-lo amb la classe nova.
42          try (ObjectInputStream in = new
ObjectInputStream(Files.newInputStream(fitxer))) {
43              Object o = in.readObject();
44              System.out.println("Llegit: " + o);
45          } catch (InvalidClassException e) {
46              System.out.println("InvalidClassException: " + e.getMessage());
47          } catch (ClassNotFoundException e) {
48              System.out.println("Classe desconeguda: " + e.getMessage());
49          }
50      }
51  }
```

Línia	Codi	Què fa i per què
15-18	`static class PerfilV1 implements Serializable { p…`	`PerfilV1`: la classe «antiga», versió 1, amb un sol camp. Classe interna `static` perquè tot càpiga en un fitxer.

> **Sortida esperada**
> ```
> InvalidClassException:
> cat.pratfp.gamevault.sessio6.Ex2VersioIncompatible$PerfilV2;
> local class incompatible: stream classdesc serialVersionUID = 1, local class
> serialVersionUID = 2
> ```

Prova-ho també a l'inrevés: deixa el `serialVersionUID` a 1 a les dues classes i torna a executar. Java **sí** que llegeix el fitxer i posa `nivell` a 0: mantenir el número és la manera de dir «aquesta versió és compatible amb l'anterior». Per això el decideixes tu.

4. Exemple 3 · Riscos: per què no s'ha de desserialitzar qualsevol cosa

`readObject` no comprova què hi ha dins del fitxer abans de crear-ho: construeix **qualsevol classe** que trobi al «classpath». Si el fitxer ve d'un usuari o de la xarxa, algú hi pot posar objectes pensats per fer coses en crear-se. És una de les vulnerabilitats més explotades de Java. Des de Java 9 hi ha una defensa senzilla: el filtre de desserialització.



Sense filtre, el fitxer mana. Amb filtre, només entren les classes que has autoritzat.

src/main/java/cat/pratfp/gamevault/sessio6/Ex3FiltreDeserialitzacio.java

```
 1  package cat.pratfp.gamevault.sessio6;
 2
 3  import java.io.IOException;
 4  import java.io.InvalidClassException;
 5  import java.io.ObjectInputFilter;
 6  import java.io.ObjectInputStream;
 7  import java.io.ObjectOutputStream;
 8  import java.nio.file.Files;
 9  import java.nio.file.Path;
10  import java.util.HashMap;
11
12  /** Exemple 3: el risc de desserialitzar qualsevol cosa, i com posar-hi un
filtre. */
13  public class Ex3FiltreDeserialitzacio {
14
15      public static void main(String[] args) throws IOException,
ClassNotFoundException {
16          Path fitxer = Path.of("dades", "sospitos.ser");
17
18          // 1. Algú ens envia un fitxer .ser. Diu que és una Biblioteca...
però hi ha un HashMap.
19          try (ObjectOutputStream out = new
ObjectOutputStream(Files.newOutputStream(fitxer))) {
20              out.writeObject(new HashMap<String, String>());
21          }
22
23          // 2. SENSE FILTRE: readObject crea el que hi hagi dins, sigui el
que sigui.
24          try (ObjectInputStream in = new
ObjectInputStream(Files.newInputStream(fitxer))) {
25              Object o = in.readObject();
26              System.out.println("Sense filtre s'ha creat un " +
o.getClass().getName());
27          }
28
29          // 3. AMB FILTRE: només acceptem les classes del nostre paquet; la
resta es rebutja.
30          ObjectInputFilter filtre = ObjectInputFilter.Config.createFilter(
31                  "cat.pratfp.gamevault.**;java.util.ArrayList;java.lang.*;!
*");
32          try (ObjectInputStream in = new
ObjectInputStream(Files.newInputStream(fitxer))) {
33              in.setObjectInputFilter(filtre);
34              in.readObject();
35          } catch (InvalidClassException e) {
36              System.out.println("Amb filtre: rebutjat -> " + e.getMessage());
37          }
38      }
39  }
```

Línia	Codi	Què fa i per què
19-21	`try (ObjectOutputStream out = new ObjectOutputStream(File…`	Simulem el fitxer «sospitós»: algú diu que és una `Biblioteca`, però hi ha un `HashMap`.
24-27	`try (ObjectInputStream in = new ObjectInputStream(Files.n…`	Sense filtre, `readObject` crea el `HashMap` sense cap queixa. El nostre programa acaba de fer cas a un fitxer extern.

Línia	Codi	Què fa i per què
30-31	`ObjectInputFilter filtre = ObjectInputFilter.Config.creat…`	El filtre és un patró de text: acceptem `cat.pratfp.gamevault.**` (el nostre codi), `ArrayList` i `java.lang.*` (les cadenes), i `!*` rebutja tota la resta. L'ordre importa: es llegeix d'esquerra a dreta.
33	`in.setObjectInputFilter(filtre);`	S'instal·la al flux **abans** de llegir.
35-36	`} catch (InvalidClassException e) { System.ou…`	El `HashMap` no passa el filtre: `InvalidClassException` amb «filter status: REJECTED». El programa no ha creat res.

Sortida esperada

```
Sense filtre s'ha creat un java.util.HashMap
Amb filtre: rebutjat -> filter status: REJECTED
```

Alternatives

La serialització Java és pràctica per a **memòria cau interna** (com la que demana l'AA3) o per guardar l'estat d'un programa entre execucions. No és un format d'intercanvi: no la llegeix cap altre llenguatge, depèn de la versió de la classe i té els riscos d'aquest exemple. Per intercanviar dades amb altres programes s'usa JSON o XML. Per això existeix la segona meitat de la sessió.

5. JSON i Jackson

JSON és text: objectes entre claus, llistes entre claudàtors, parelles clau-valor. El llegeix qualsevol llenguatge i és el format de les API REST. Jackson és la biblioteca més usada a Java per convertir objectes en JSON i a l'inrevés. Cal afegir-la al `pom.xml` amb el mòdul de dates:

pom.xml (dins de <dependencies>)

```
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.17.1</version>
</dependency>
<dependency>
    <groupId>com.fasterxml.jackson.datatype</groupId>
    <artifactId>jackson-datatype-jsr310</artifactId>
    <version>2.17.1</version>
</dependency>
```

Jackson llegeix els accessors del record (id(), titol()…) i en fa les claus del JSON



L'ObjectMapper és el traductor en els dos sentits.

Amb un `record` com `Joc`, Jackson ho té fàcil: fa servir els accessors (`id()`, `titol()` …) com a claus i el constructor canònic per reconstruir-lo. No cal cap anotació.

6. Exemple 4 · Del catàleg a JSON i tornada

src/main/java/cat/pratfp/gamevault/sessio6/Ex4JsonAmbJackson.java

```
 1  package cat.pratfp.gamevault.sessio6;
 2
 3  import cat.pratfp.gamevault.model.Joc;
 4  import com.fasterxml.jackson.core.type.TypeReference;
 5  import com.fasterxml.jackson.databind.DeserializationFeature;
 6  import com.fasterxml.jackson.databind.ObjectMapper;
 7  import com.fasterxml.jackson.databind.SerializationFeature;
 8  import com.fasterxml.jackson.datatype.jsr310.JavaTimeModule;
 9
10  import java.io.IOException;
11  import java.nio.charset.StandardCharsets;
12  import java.nio.file.Files;
13  import java.nio.file.Path;
14  import java.time.LocalDate;
15  import java.util.List;
16
17  /** Exemple 4: escriure i llegir una llista de Joc en JSON amb Jackson. */
18  public class Ex4JsonAmbJackson {
19
20      public static void main(String[] args) throws IOException {
21          Path json = Path.of("dades", "export", "jocs.json");
22          Files.createDirectories(json.getParent());
23
24          // 1. L'ObjectMapper és la peça central. El configurem un sol cop.
25          ObjectMapper mapper = new ObjectMapper()
26                  .registerModule(new
JavaTimeModule())                       // entén LocalDate
27                  .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS)
// "2017-02-24" i no [2017,2,24]
28                  .disable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES);
// tolera camps de més
29
30          List<Joc> jocs = List.of(
31                  new Joc(1, "Hollow Knight", "PC", "Team Cherry",
LocalDate.of(2017, 2, 24), 42.5, 9.5),
32                  new Joc(4, "Gris", "PC", "Nomada Studio", LocalDate.of(2018,
12, 13), 4.0, 8.0));
33
34          // 2. ESCRIURE: de la llista d'objectes a text JSON indentat.
35          mapper.writerWithDefaultPrettyPrinter().writeValue(json.toFile(),
jocs);
36          System.out.println(Files.readString(json, StandardCharsets.UTF_8));
37
38          // 3. LLEGIR: del text JSON a una List<Joc>. TypeReference conserva
el tipus de la llista.
39          List<Joc> llegits = mapper.readValue(json.toFile(), new
TypeReference<List<Joc>>() {});
40          System.out.println("Llegits " + llegits.size() + " jocs. El primer:
" + llegits.get(0).titol()
41                  + " (" + llegits.get(0).dataSortida().getYear() + ")");
42          System.out.println("Iguals als originals? " + llegits.equals(jocs));
43      }
44  }
```

Línia	Codi	Què fa i per què
25-28	`ObjectMapper mapper = new ObjectMapper()` …	La configuració de l'`ObjectMapper`, que has de repetir a l'AA3. `JavaTimeModule` fa que Jackson entengui `LocalDate`; `WRITE_DATES_AS_TIMESTAMPS` desactivat escriu `"2017-02-24"` en lloc de `[2017,2,24]`; `FAIL_ON_UNKNOWN_PROPERTIES` desactivat tolera claus de més en llegir.
30-32	`List<Joc> jocs = List.of( new Joc(1, "Hol…`	Dos jocs, amb el `record Joc` del paquet `model` (el mateix de l'AA2).
35	`mapper.writerWithDefaultPrettyPrinter().writeValue(json.t…`	`writerWithDefaultPrettyPrinter()` indenta la sortida; `writeValue(File, Object)` escriu la llista sencera.
36	`System.out.println(Files.readString(json, StandardCharset…`	Mostrem el text tal com ha quedat al fitxer.
39	`List<Joc> llegits = mapper.readValue(json.toFile(), new T…`	El camí de tornada. Com que `List<Joc>` és un tipus genèric i Java l'esborra en compilar, cal dir-li a Jackson quin és amb `new TypeReference<List<Joc>>() {}`. Sense això, obtindries una llista de `LinkedHashMap`.
40-41	`System.out.println("Llegits " + llegits.size() + " jocs. …`	La data ha tornat com a `LocalDate` de veritat: `getYear()` funciona.
42	`System.out.println("Iguals als originals? " + llegits.equ…`	`equals` d'un record compara camp a camp: la llista llegida és igual a l'original. Anada i tornada sense pèrdua.

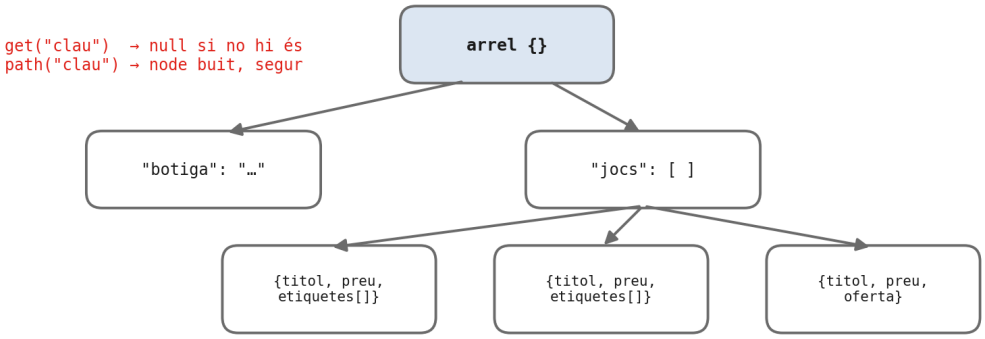
Sortida esperada (retallada)

```
[ {
  "id" : 1,
  "titol" : "Hollow Knight",
  "plataforma" : "PC",
  "estudi" : "Team Cherry",
  "dataSortida" : "2017-02-24",
  "horesJugades" : 42.5,
  "nota" : 9.5
}, {
  "id" : 4,
  ...
} ]
Llegits 2 jocs. El primer: Hollow Knight (2017)
Iguals als originals? true
```

Compte amb això
Si oblides `registerModule(new JavaTimeModule())`, el primer `writeValue` peta amb «Java 8 date/time type `java.time.LocalDate` not supported by default». És l'error més habitual de l'AA3 i el missatge et diu exactament què falta.

7. Exemple 5 · Llegir JSON sense cap model: JsonNode

De vegades el JSON no és teu (ve d'una API) i no vols crear una classe per a cada resposta. `readTree` converteix el text en un arbre de nodes que pots recórrer com vulguis. És el que demana el mètode `resumeixSenseModel` de l'AA3.



L'arbre de nodes d'un JSON. Cada clau és un fill; cada element d'un array, també.

src/main/java/cat/pratfp/gamevault/sessio6/Ex5JsonSenseModel.java

```
 1  package cat.pratfp.gamevault.sessio6;
 2
 3  import com.fasterxml.jackson.databind.JsonNode;
 4  import com.fasterxml.jackson.databind.ObjectMapper;
 5
 6  import java.io.IOException;
 7  import java.nio.charset.StandardCharsets;
 8  import java.nio.file.Files;
 9  import java.nio.file.Path;
10
11  /** Exemple 5: recórrer un JSON amb JsonNode, sense cap classe Java al
darrere. */
12  public class Ex5JsonSenseModel {
13
14      static final String RESPOSTA_API = """
15              {
16                "botiga": "GameVault Store",
17                "jocs": [
18                  {"titol": "Hollow Knight", "preu": 14.99, "etiquetes":
["metroidvania", "indie"]},
19                  {"titol": "Celeste", "preu": 19.99, "etiquetes":
["plataformes"]},
20                  {"titol": "Gris", "preu": 16.99, "oferta": true}
21                ]
22              }
23              """;
24
25      public static void main(String[] args) throws IOException {
26          Path fitxer = Path.of("dades", "import", "botiga.json");
27          Files.createDirectories(fitxer.getParent());
28          Files.writeString(fitxer, RESPOSTA_API, StandardCharsets.UTF_8);
29
30          // 1. readTree converteix el text en un arbre de nodes.
31          JsonNode arrel = new ObjectMapper().readTree(fitxer.toFile());
32          System.out.println("Botiga: " + arrel.get("botiga").asText());
33
34          // 2. Recorrem l'array "jocs" node a node.
35          double total = 0;
36          for (JsonNode joc : arrel.get("jocs")) {
37              String titol = joc.get("titol").asText();
38              double preu = joc.get("preu").asDouble();
39              // path() no peta si la clau no existeix: retorna un node
"missing".
40              boolean oferta = joc.path("oferta").asBoolean(false);
41              int etiquetes = joc.path("etiquetes").size();
42              System.out.printf("%-15s %6.2f € %s (%d etiquetes)%n", titol,
preu, oferta ? "OFERTA" : "", etiquetes);
43              total += preu;
44          }
45          System.out.printf("Total: %.2f €%n", total);
46      }
47  }
```

Línia	Codi	Què fa i per què

14-23	`static final String RESPOSTA_API = """`
	`{`
	`…`
		Una resposta d'API simulada. Fixa't que els tres jocs **no tenen els mateixos camps**: dos tenen `etiquetes`, un té `oferta`. Amb una classe fixa això seria incòmode.

31	`JsonNode arrel = new ObjectMapper().readTree(fitxer.toFil…`	readTree retorna el node arrel. No hem configurat res: per a un arbre no calen mòduls.

Línia	Codi	Què fa i per què
32	`System.out.println("Botiga: " + arrel.get("botiga").asTex…`	`get("botiga")` baixa un nivell; `asText()` en treu el valor com a cadena.
36	`for (JsonNode joc : arrel.get("jocs")) {`	Un `JsonNode` que és un array es pot recórrer amb `for-each`: cada volta és un joc.
37-38	`String titol = joc.get("titol").asText(); dou…`	`asText()` i `asDouble()` converteixen el valor del node al tipus que vols.
40	`boolean oferta = joc.path("oferta").asBoolean(false);`	`path` en lloc de `get`: si la clau no hi és, retorna un node «missing» en lloc de `null`, i `asBoolean(false)` dona el valor per defecte. Cap `NullPointerException`.
41	`int etiquetes = joc.path("etiquetes").size();`	`size()` d'un node array és el nombre d'elements; d'un node «missing», 0.

Sortida esperada

```
Botiga: GameVault Store
Hollow Knight    14.99 €  (2 etiquetes)
Celeste          19.99 €  (1 etiquetes)
Gris             16.99 € OFERTA (0 etiquetes)
Total: 51.97 €
```

	`readValue` amb classe	`readTree` amb `JsonNode`
Quan	El JSON és estable i és teu (el catàleg)	El JSON ve de fora, canvia o només en vols un tros
Què obtens	Objectes Java amb tipus (`List<Joc>`)	Un arbre genèric que recorres a mà
Errors	En compilar, si t'equivoques de camp	En executar, si la clau no existeix (`get` → `null`)
A l'AA3	`JsonJocDao.llegeix`	`resumeixSenseModel`

8. Resum de classes i mètodes d'avui

Classe	Mètodes clau	Quan la fas servir
`Serializable`	— (interfície marca)	Qualsevol classe que vulguis guardar sencera amb Java.
`ObjectOutputStream` / `ObjectInputStream`	`writeObject`, `readObject`	Desar i recuperar objectes. Sempre amb `try-with-resources`.
`serialVersionUID`	`private static final long`	Controlar la compatibilitat entre versions de la classe.
`transient`	—	Camps que no s'han de guardar (derivats, recursos, secrets).
`ObjectInputFilter`	`Config.createFilter`, `setObjectInputFilter`	Limitar quines classes es poden desserialitzar.

Classe	Mètodes clau	Quan la fas servir
`ObjectMapper`	`registerModule`, `disable`, `writeValue`, `readValue`, `readTree`	El traductor objecte ↔ JSON de Jackson.
`TypeReference<T>`	`new TypeReference<List<Joc>>() {}`	Dir-li a Jackson el tipus genèric que vols en llegir.
`JsonNode`	`get`, `path`, `asText`, `asDouble`, `asBoolean`, `size`	Recórrer JSON sense model.

9. Relació amb l'AA3 «Canvia de format»

Punt de l'AA3	Classe que has de fer	Exemple que t'hi ajuda
1 · Dependències al pom.xml	—	Apartat 5 (Jackson). Les de JAXB, a la sessió 7
2 · Serialització Java + experiment	`Biblioteca` amb `desa` i `carrega`	Exemples 1 i 2
3 · JSON amb Jackson	`JsonJocDao` i `resumeixSenseModel`	Exemples 4 i 5
4-6 · XML amb JAXB, adaptadors, convertidor	`XmlJocDao`, `Convertidor`	Sessió 7
9 · Excepcions controlades	`CatalegException`	Exemple 3 (captura d'`InvalidClassException`) i el patró de la sessió 5

L'AA3 és **en parella** i es lliura el dimarts 06/10 a les 23:59. Avui hauríeu de deixar `Biblioteca` i `JsonJocDao` fets, amb les proves JUnit de serialització i d'anada i tornada JSON en verd, i l'`InvalidClassException` capturada i documentada.

10. Problemes típics i solució

Símptoma	Causa probable	Solució
`NotSerializableException: ...Joc`	Un camp de la classe apunta a un objecte no serialitzable	Fes que aquella classe també implementi `Serializable`, o marca el camp `transient`.
`InvalidClassException: local class incompatible`	El `serialVersionUID` del fitxer i el de la classe no coincideixen	És l'experiment del punt 2. En producció: no canviïs el número si vols mantenir la compatibilitat.
`ClassCastException` en fer el «cast» de `readObject`	El fitxer no conté la classe que esperaves	Comprova `instanceof` abans del «cast» i posa un filtre.
«Java 8 date/time type not supported by default»	Falta el `JavaTimeModule`	`registerModule(new JavaTimeModule())` i la dependència `jackson-datatype-jsr310`.
La data surt com `[2017,2,24]`	`WRITE_DATES_AS_TIMESTAMPS` activat	`disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS)`.

Símptoma	Causa probable	Solució
`readValue` retorna `List<LinkedHashMap>`	Has passat `List.class` en lloc del tipus genèric	Usa `new TypeReference<List<Joc>>() {}`.
`UnrecognizedPropertyException`	El JSON té una clau que `Joc` no té	`disable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES)`.
`NullPointerException` en recórrer un `JsonNode`	`get` d'una clau que no existeix	Usa `path(...)` i els `asXxx(valorPerDefecte)`.

11. Comprova que ho has entès

1. Per què `Serializable` no té cap mètode i tot i així «fa» alguna cosa?
2. Després de carregar una `Biblioteca` d'un `.ser`, `getHoresCalculades()` retorna 0.0. És un error?
3. Si afegeixes un camp a una classe però **no** canvies el `serialVersionUID`, es poden llegir els fitxers antics?
4. Què fa el patró `"cat.pratfp.gamevault.**;!*"` en un `ObjectInputFilter`?
5. Per què `readValue(fitxer, List.class)` no serveix per obtenir una `List<Joc>`?
6. Quina diferència hi ha entre `node.get("oferta")` i `node.path("oferta")` si la clau no existeix?

Solucions

- 1. És una interfície «marca»: Java la comprova amb `instanceof` dins de `writeObject` per decidir si pot serialitzar l'objecte.
- 2. No: el camp és `transient` i no es guarda. Si el necessites, recalcula'l en carregar (o no el facis `transient`).
- 3. Sí. Java llegeix el fitxer i deixa el camp nou amb el valor per defecte. Mantenir el número és declarar la compatibilitat.
- 4. Permet qualsevol classe del nostre paquet i subpaquets, i rebutja qualsevol altra. Els patrons es llegeixen d'esquerra a dreta.
- 5. Perquè el tipus genèric s'esborra en compilar: Jackson no sap què hi ha dins de la llista i crea `LinkedHashMap`. Cal `TypeReference`.
- 6. `get` retorna `null` (i el següent `.asText()` peta); `path` retorna un node «missing» amb el qual `asBoolean(false)` funciona.

12. Per a la propera sessió

Dimarts 06/10/2026 · AA3 (2a part): XML amb JAXB (DTO amb constructor buit, `@XmlRootElement`, `@XmlAttribute`), els adaptadors amb Stream i el convertidor universal per extensió. Porta `Biblioteca` i `JsonJocDao` acabats: el convertidor els necessita. Ves a la documentació de JAXB (eclipse-ee4j.github.io/jaxb-ri) i mira què és el «marshalling».

Bibliografia/Webgrafia

- Oracle. Java Object Serialization Specification (Java SE 21). docs.oracle.com/en/java/javase/21/docs/specs/serialization/
- Oracle. Java SE 21 API: `java.io.ObjectInputFilter`. docs.oracle.com/en/java/javase/21/docs/api/
- Bloch, J. Effective Java (3a ed.), capítol 12: «Serialization» (ítems 85-90).

- FasterXML. Jackson Databind: documentació i wiki. github.com/FasterXML/jackson-databind
- FasterXML. Jackson modules for Java 8 date/time. github.com/FasterXML/jackson-modules-java8
- ECMA-404. The JSON Data Interchange Syntax. json.org
- Material del mòdul: apunts AEA1 «Gestió de fitxers amb Java», apartats 5 i 6, i fitxa de l'AA3 (Classroom).
- Repositori d'exemples: github.com/ilopez000/0486-acces-a-dades

Data actualització i autor/res

Ignasi López Aylagas · 29/09/2026